

✓ EECS 242 Advanced Topics in Deep Learning - Assignment 2

Due at **11:59PM October 11th, 2025**

Put your answers in solution sections, add missing code, run all code blocks.

Make sure you choose a runtime type with a GPU, e.g., T4 GPU. You don't need to subscribe to Colab Pro for such runtime type.

What to submit: Print the notebook with output, save as a PDF, and submit the PDF to CatCourse.

It is very important you have output shown in the PDF submitted.

Save a copy of this notebook in drive so you don't lose your work.

✓ Part I: Convolutional Neural Network (3 points)

1. [2 points] Consider the convolutional neural network defined by the layers in the left column below. Fill in the shape of the output volume and the number of parameters at each layer. You can write the activation shapes in the format (H, W, C) for 3D volume, where H, W, C are the height, width and channel dimensions, respectively. Unless specified, assume padding 1, stride 1 where appropriate.

Notation:

- CONVx-N denotes a convolutional layer with N filters with kernel height and width equal to x
- POOL-N denotes a $N \times N$ max-pooling layer with stride of N and 0 padding.
- FLATTEN flattens its inputs, identical to `torch.nn.flatten` / `tf.layers.flatten`
- FC-N denotes a fully-connected layer with N neurons

Fill out the table bellow:



Soluton:

Layer	Activation volume dimensions	Number of parameters (weights and biases)
Input	(32, 32, 3)	0
Conv3-8	(32, 32, 8)	weight:3x3x3x8=216, bias:8, total= 224
ReLU	(32, 32, 8)	0
POOL-2	(16, 16, 8)	0
BATCHNORM	(16, 16, 8)	2x8= 16
Conv3-16	(16, 16, 16)	weight:3x3x8x16=1152, bias:16, total= 1168
ReLU	(16, 16, 16)	0
POOL-2	(8, 8, 16)	0
FLATTEN	(1024, 1, 1)	0
FC-10	(10, 1, 1)	weight:1024x10=10240, bias:10, total= 10250

2. [1 point] What types of losses are used to train an object detection network? Notation is not needed here. Just describe the losses in two to four sentences.

Soluton:

For classification, we usually use cross-entropy loss, which measures the difference between the ground-truth label and the predicted probability distribution. (We may apply softmax function to transform the output score into probability distribution.)

For localization, we usually use L1, L2, IoU losses to measure the difference between the ground-truth bounding box coordinates and the predicted coordinates.

3. [1 point] What is the goal of dilated convolution (a.k.a. atrous convolution)? Notation is not needed here. Answer in two to four sentences.

Soluton:

Increase the receptive field with the same kernel size by inserting holes (ignoring some pixels).

✓ Part II: Open-vocabulary Image Classification (3 points)

4. In this part, you are supposed to use CLIP model to build an open-vocabulary image classification.

Related lecture: Lecture 8 Image Segmentation Part II

✓ Preparation for Colab

Make sure you're running a GPU runtime; if not, select "GPU" as the hardware accelerator in Runtime > Change Runtime Type in the menu. The next cells will install the `clip` package and its dependencies, and check if PyTorch 1.7.1 or later is installed.

```
! pip install ftfy regex tqdm
! pip install git+https://github.com/openai/CLIP.git
```

Collecting ftfy

Downloading ftfy-6.3.1-py3-none-any.whl.metadata (7.3 kB)

Requirement already satisfied: regex in /usr/local/lib/python3.12/dist-packages (2024.11.6)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.1)

Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-packages (from ftfy) (0.2.14)

Downloading ftfy-6.3.1-py3-none-any.whl (44 kB)

44.8/44.8 kB 2.4 MB/s eta 0:00:00

Installing collected packages: ftfy

Successfully installed ftfy-6.3.1

Collecting git+<https://github.com/openai/CLIP.git>

Cloning <https://github.com/openai/CLIP.git> to /tmp/pip-req-build-z8vgp1pp

Running command git clone --filter=blob:none --quiet <https://github.com/openai/CLIP.git> /tmp/pip-req-build

Resolved <https://github.com/openai/CLIP.git> to commit dcba3cb2e2827b402d2701e7e1c7d9fed8a20ef1

```
Preparing metadata (setup.py) ... done
Requirement already satisfied: ftfy in /usr/local/lib/python3.12/dist-packages (from clip==1.0) (6.3.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from clip==1.0) (25.0)
Requirement already satisfied: regex in /usr/local/lib/python3.12/dist-packages (from clip==1.0) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from clip==1.0) (4.67.1)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (from clip==1.0) (2.8.0+cu1
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (from clip==1.0) (0.2
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-packages (from ftfy->clip==1.0) (0
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from t
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch->clip==
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (3
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch->clip==1.0) (2
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: nvidia-cuspars-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from t
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch->clip==
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision->clip==1.
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torch
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->tor
Building wheels for collected packages: clip
  Building wheel for clip (setup.py) ... done
  Created wheel for clip: filename=clip-1.0-py3-none-any.whl size=1369490 sha256=1141e21b92b61ba2494d0c1ff2
  Stored in directory: /tmp/pip-ephem-wheel-cache-6y2_lzsc/wheels/35/3e/df/3d24cbfb3b6a06f17a2bfd7d1138900d
Successfully built clip
```

```
Installing collected packages: clip  
Successfully installed clip-1.0
```

```
import numpy as np  
import torch  
from pkg_resources import packaging  
  
print("Torch version:", torch.__version__)
```

```
Torch version: 2.8.0+cu126  
/tmp/ipython-input-2880966951.py:3: DeprecationWarning: pkg_resources is deprecated as an API. See https://  
    from pkg_resources import packaging
```

✓ Loading the model

`clip.available_models()` will list the names of available CLIP models.

```
import clip  
  
clip.available_models()
```

```
['RN50',  
 'RN101',  
 'RN50x4',  
 'RN50x16',  
 'RN50x64',  
 'ViT-B/32',  
 'ViT-B/16',  
 'ViT-L/14',  
 'ViT-L/14@336px']
```

```
# We will use CLIP with ViT-B/32 backbone in this assignment
model, preprocess = clip.load("ViT-B/32")
model.cuda().eval()
input_resolution = model.visual.input_resolution
context_length = model.context_length
vocab_size = model.vocab_size

print("Model parameters:", f"{np.sum([int(np.prod(p.shape)) for p in model.parameters()]):,}")
print("Input resolution:", input_resolution)
print("Context length:", context_length)
print("Vocab size:", vocab_size)
```

```
100%|████████████████████████████████████████| 338M/338M [00:06<00:00, 57.8MiB/s]
Model parameters: 151,277,313
Input resolution: 224
Context length: 77
Vocab size: 49408
```

✓ Image Preprocessing

The second return value from `clip.load()` contains a torchvision `Transform` that performs this preprocessing.

preprocess

```
Compose(
  Resize(size=224, interpolation=bicubic, max_size=None, antialias=True)
  CenterCrop(size=(224, 224))
  <function _convert_image_to_rgb at 0x7963f189c7c0>
  ToTensor()
  Normalize(mean=(0.48145466, 0.4578275, 0.40821073), std=(0.26862954, 0.26130258, 0.27577711))
)
```

4.1 [1 point] What are the image processing steps included in image preprocess above?

Soluton:

1. Scale the image to let the shorter side be 224, for example, a 400x300 image will be resized to 298x224
2. Crop a 224x224 image from the center of the input image
3. Convert the image to RGB
4. Convert it to Pytorch tensor, that is, transform the pixel range to [0,1]
5. Makes the pixel roughly zero-mean and unit variance

▼ Text Preprocessing

We use a case-insensitive tokenizer, which can be invoked using `clip.tokenize()`. By default, the outputs are padded to become 77 tokens long, which is what the CLIP models expects.

```
clip.tokenize("Hello World!")
```

[illegible]

✓ Setting up input images and texts

We are going to feed 8 example images and their textual descriptions to the model, and compare the similarity between the corresponding features.

The tokenizer is case-insensitive, and we can freely give any suitable textual descriptions.

```
import os
import skimage
from skimage import data

import IPython.display
import matplotlib.pyplot as plt
from PIL import Image
import numpy as np

from collections import OrderedDict
import torch

%matplotlib inline
%config InlineBackend.figure_format = 'retina'

# images in skimage to use and their textual descriptions
descriptions = {
    "page": "a page of text about segmentation",
    "chelsea": "a facial photo of a tabby cat",
    "astronaut": "a portrait of an astronaut with the American flag",
    "rocket": "a rocket standing on a launchpad",
    "motorcycle_right": "a red motorcycle standing in a garage",
    "camera": "a person looking at a camera on a tripod",
    "horse": "a black-and-white silhouette of a horse",
    "coffee": "a cup of coffee on a saucer"
}
```



```

_ = data.download_all()

original_images = []
images = []
texts = []
plt.figure(figsize=(16, 5))

for filename in [filename for filename in os.listdir(skimage.data_dir) if filename.endswith(".png") or filename.endswith(".jpg")]:
    name = os.path.splitext(filename)[0]
    if name not in descriptions:
        continue

    image = Image.open(os.path.join(skimage.data_dir, filename)).convert("RGB")

    plt.subplot(2, 4, len(images) + 1)
    plt.imshow(image)
    plt.title(f"{filename}\n{descriptions[name]}")
    plt.xticks([])
    plt.yticks([])

    original_images.append(image)
    images.append(preprocess(image))
    texts.append(descriptions[name])

plt.tight_layout()

```

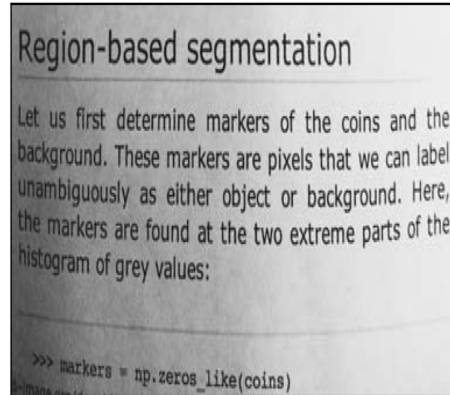
```

Downloading file 'color/tests/data/lab_array_a_10.npy' from 'https://github.com/scikit-image/scikit-image/raw/master/color/tests/data/lab_array_a_10.npy'
Downloading file 'color/tests/data/lab_array_a_2.npy' from 'https://github.com/scikit-image/scikit-image/raw/master/color/tests/data/lab_array_a_2.npy'
Downloading file 'color/tests/data/lab_array_a_r.npy' from 'https://github.com/scikit-image/scikit-image/raw/master/color/tests/data/lab_array_a_r.npy'
Downloading file 'color/tests/data/lab_array_b_10.npy' from 'https://github.com/scikit-image/scikit-image/raw/master/color/tests/data/lab_array_b_10.npy'
Downloading file 'color/tests/data/lab_array_b_2.npy' from 'https://github.com/scikit-image/scikit-image/raw/master/color/tests/data/lab_array_b_2.npy'
Downloading file 'color/tests/data/lab_array_b_r.npy' from 'https://github.com/scikit-image/scikit-image/raw/master/color/tests/data/lab_array_b_r.npy'
Downloading file 'color/tests/data/lab_array_c_10.npy' from 'https://github.com/scikit-image/scikit-image/raw/master/color/tests/data/lab_array_c_10.npy'
Downloading file 'color/tests/data/lab_array_c_2.npy' from 'https://github.com/scikit-image/scikit-image/raw/master/color/tests/data/lab_array_c_2.npy'
Downloading file 'color/tests/data/lab_array_c_r.npy' from 'https://github.com/scikit-image/scikit-image/raw/master/color/tests/data/lab_array_c_r.npy'

```

[illegible]

page.png
a page of text about segmentation



chelsea.png
a facial photo of a tabby cat



motorcycle_right.png
a red motorcycle standing in a garage



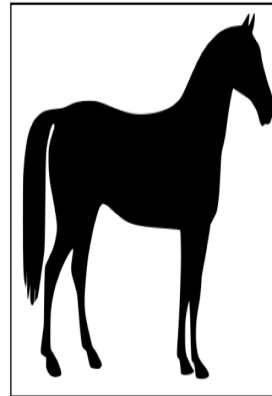
astronaut.png
a portrait of an astronaut with the American flag



rocket.jpg
a rocket standing on a launchpad



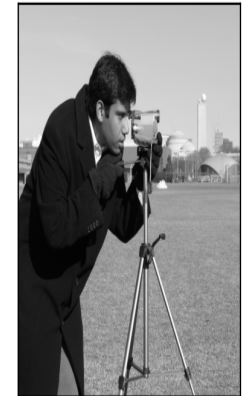
horse.png
a black-and-white silhouette of a horse



coffee.png
a cup of coffee on a saucer



camera.png
a person looking at a camera on a tripod



✓ Building features

We normalize the images, tokenize each text input, and run the forward pass of the model to get the image and text features.

```
image_input = torch.tensor(np.stack(images)).cuda()
text_tokens = clip.tokenize(["This is " + desc for desc in texts]).cuda()
```

```
with torch.no_grad():
    image_features = model.encode_image(image_input).float()
    text_features = model.encode_text(text_tokens).float()
```

4.2 [1 point] Fill missing code bellow to compute similarity between image embedding and text embedding. First normalize image features and text features to be unit norm. Then take the dot product between text features and image faetures.

✓ Calculating cosine similarity

We normalize the features and calculate the dot product of each pair.

```
image_features /= image_features.norm(dim=1, keepdim=True)
text_features /= text_features.norm(dim=1, keepdim=True)
similarity = torch.matmul(image_features, text_features.T).cpu().numpy()

count = len(descriptions)

plt.figure(figsize=(20, 14))
plt.imshow(similarity, vmin=0.1, vmax=0.3)
# plt.colorbar()
plt.yticks(range(count), texts, fontsize=18)
plt.xticks([])
for i, image in enumerate(original_images):
    plt.imshow(image, extent=(i - 0.5, i + 0.5, -1.6, -0.6), origin="lower")
for x in range(similarity.shape[1]):
    for y in range(similarity.shape[0]):
        plt.text(x, y, f"{similarity[y, x]:.2f}", ha="center", va="center", size=12)
```

```
for side in ["left", "top", "right", "bottom"]:
    plt.gca().spines[side].set_visible(False)

plt.xlim([-0.5, count - 0.5])
plt.ylim([count + 0.5, -2])

plt.title("Cosine similarity between text and image features", size=20)
```

```
Text(0.5, 1.0, 'Cosine similarity between text and image features')
```

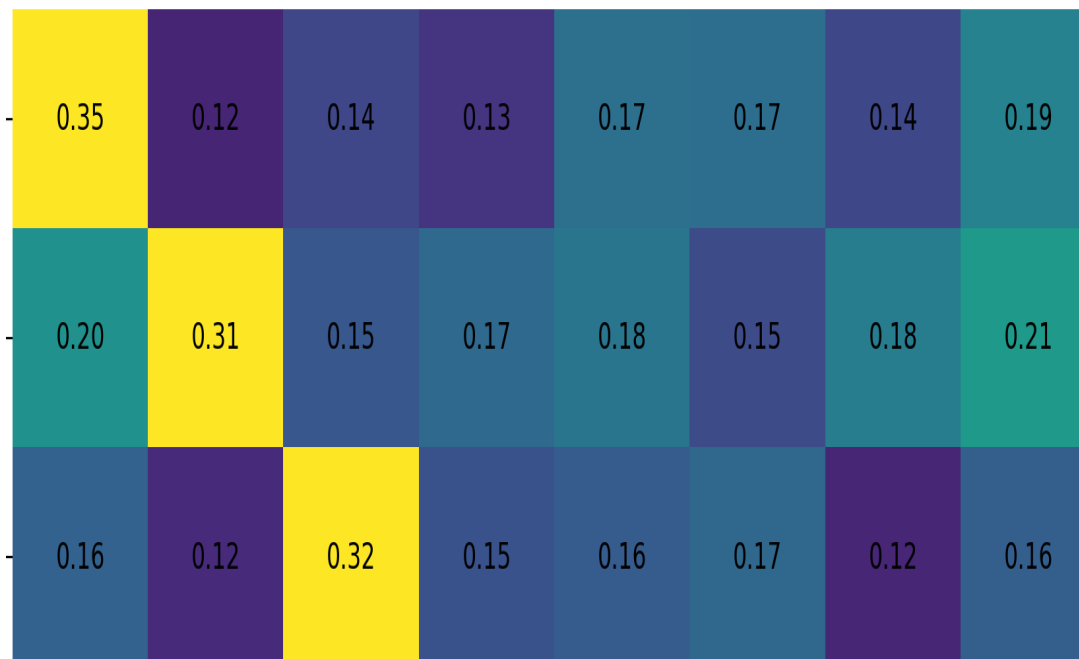
Cosine similarity between text and image features



a page of text about segmentation

a facial photo of a tabby cat

a red motorcycle standing in a garage



a portrait of an astronaut with the American flag

0.15 0.12 0.15 0.28 0.19 0.11 0.15 0.19

a rocket standing on a launchpad

0.16 0.12 0.16 0.22 0.30 0.15 0.12 0.21

a black-and-white silhouette of a horse

0.20 0.15 0.16 0.16 0.17 0.35 0.15 0.20

a cup of coffee on a saucer

0.20 0.17 0.13 0.15 0.14 0.15 0.29 0.14

a person looking at a camera on a tripod

0.20 0.16 0.12 0.17 0.20 0.21 0.17 0.30

✓ Zero-Shot Image Classification

You can classify images using the cosine similarity (times 100) as the logits to the softmax operation.

```
from torchvision.datasets import CIFAR100

cifar100 = CIFAR100(os.path.expanduser("~/cache"), transform=preprocess, download=True)

100%|██████████| 169M/169M [00:03<00:00, 43.3MB/s]
```

4.3 [1 point] Complete code bellow for zero-shot image classification

```
text_descriptions = [f"This is a photo of a {label}" for label in cifar100.classes]

# Tokenize text descriptions
text_tokens = clip.tokenize(text_descriptions).cuda()
with torch.no_grad():
    #encode text
    text_features = model.encode_text(text_tokens).float()
    #normalized text features
    text_features /= text_features.norm(dim=1, keepdim=True)
#compute dot product between pairs of image feature and text feature
similarity = torch.matmul(image_features, text_features.T)

text_probs = (100.0 * similarity).softmax(dim=-1)
top_probs, top_labels = text_probs.cpu().topk(5, dim=-1)
```

```
# visualize classification results
plt.figure(figsize=(16, 16))

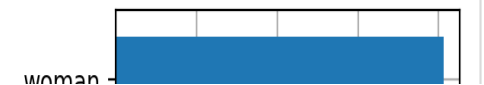
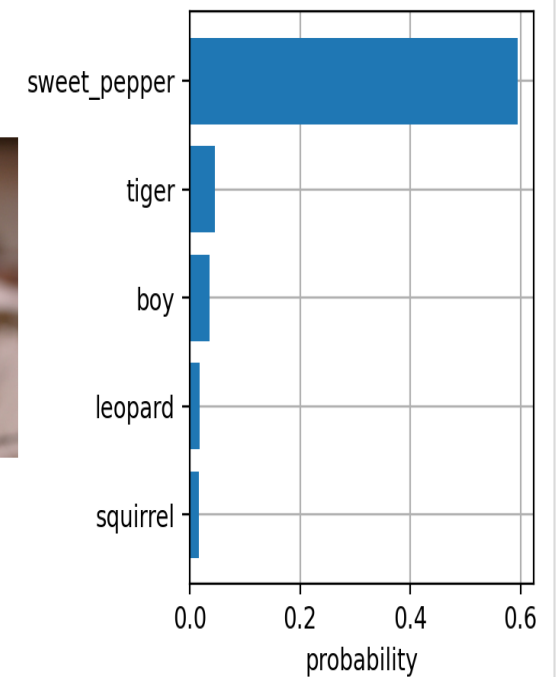
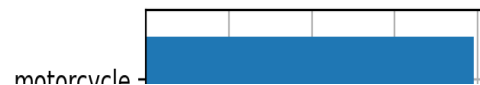
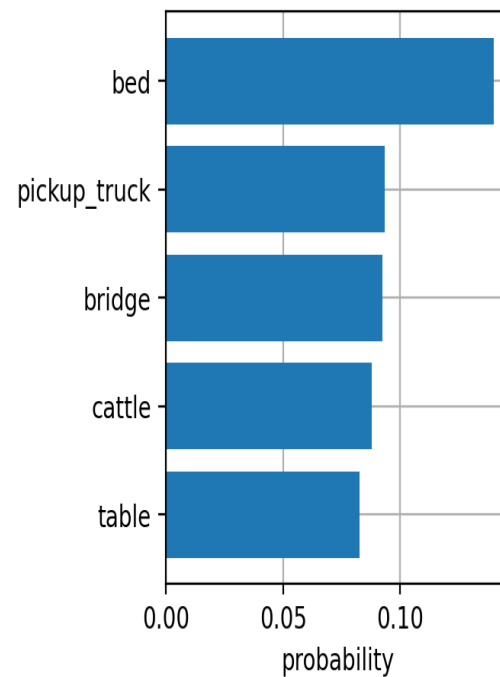
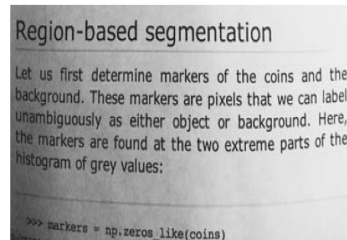
for i, image in enumerate(original_images):
```

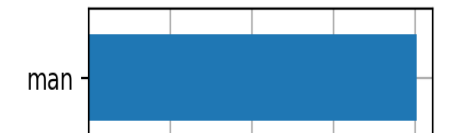
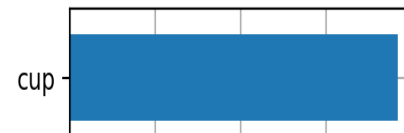
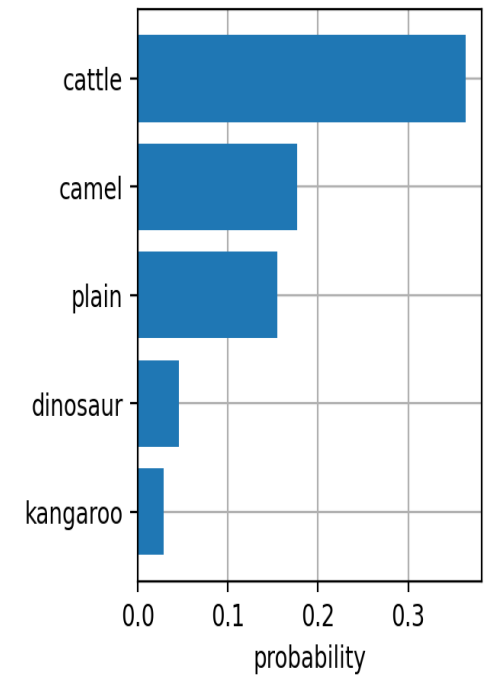
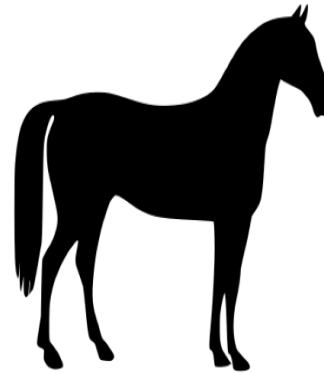
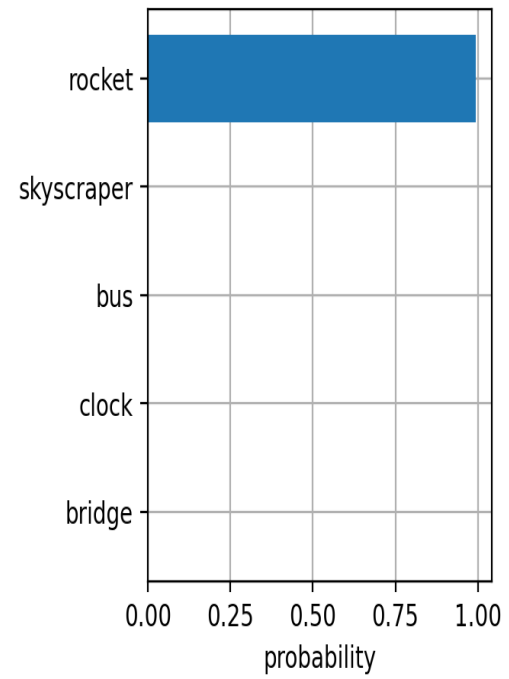
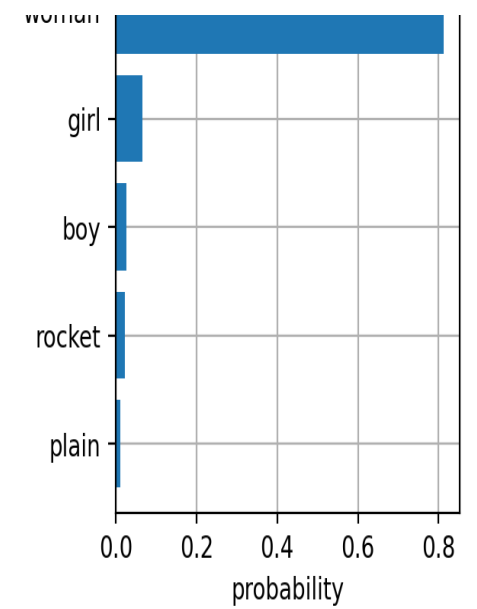


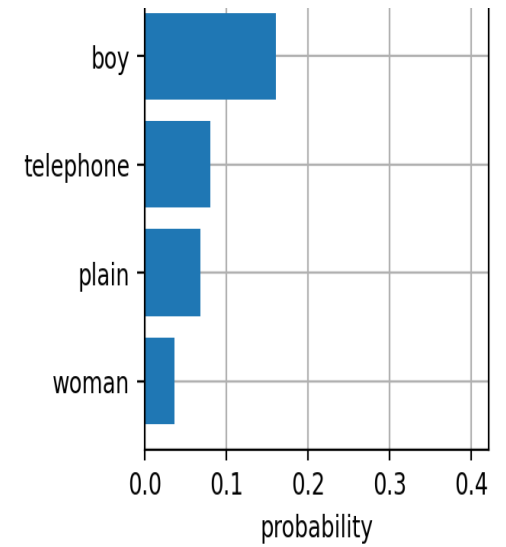
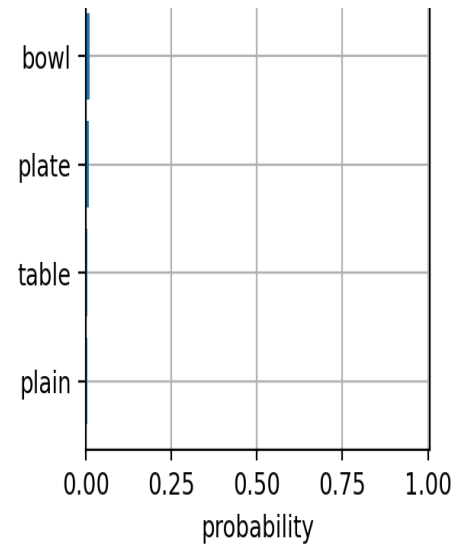
```
plt.subplot(4, 4, 2 * i + 1)
plt.imshow(image)
plt.axis("off")
```

```
plt.subplot(4, 4, 2 * i + 2)
y = np.arange(top_probs.shape[-1])
plt.grid()
plt.barh(y, top_probs[i])
plt.gca().invert_yaxis()
plt.gca().set_axisbelow(True)
plt.yticks(y, [cifar100.classes[index] for index in top_labels[i].numpy()])
plt.xlabel("probability")
```

```
plt.subplots_adjust(wspace=0.5)
plt.show()
```







✓ Part III: Implicit neural field (4 points)

5. In this part, you are supposed to implement a multi-layer perceptron for implicit neural field and train on ONE image only (overfitting).

Related lecture on implicit neural field: lecture 3 on Neural Network

5.1 [1 point] Implement a Multi-layer perceptron in pytorch that maps an image coordinate (x,y) to RGB color.

```
# your code here
import torch.nn as nn
class mlp(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, 128), # input (x, y)
            nn.LeakyReLU(),
            nn.BatchNorm1d(128),
            nn.Linear(128, 512),
            nn.LeakyReLU(),
            nn.BatchNorm1d(512),
            nn.Linear(512, 512),
            nn.LeakyReLU(),
            nn.BatchNorm1d(512),
            nn.Linear(512, 128),
            nn.LeakyReLU(),
            nn.BatchNorm1d(128),
            nn.Linear(128, 3), # output (r, g, b)
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.net(x)
```

5.2 [2 points] Train the MLP for one image provided at <http://www.mengtang.org/yosemite.png> , plot loss at each training iteration.

```
# get image and convert to tensor
from PIL import Image
import requests
from io import BytesIO
import torchvision.transforms as T

url = "http://www.mengtang.org/yosemite.png"
response = requests.get(url)
img = Image.open(BytesIO(response.content)).convert("RGB")

to_tensor = T.ToTensor()
img_tensor = to_tensor(img)
C, H, W = img_tensor.shape

print(C, H, W)
```

```
3 571 400
```

```
import torch
from tqdm import tqdm

# get feature xy
ys, xs = torch.meshgrid(torch.arange(H), torch.arange(W), indexing='ij')
coords = torch.stack([xs, ys], dim=-1).view(-1, 2).float()
print(f"feature shape: {coords.shape}")

# normalized to [0, 1]
coords[:, 0] /= (W - 1)
coords[:, 1] /= (H - 1)

# get label rgb
colors = img_tensor.permute(1, 2, 0).view(-1, 3)
print(f"label shape: {colors.shape}")
```

```

model = mlp()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.MSELoss()

epochs = 100
batch_size = 1024
loss_history = []

for epoch in range(epochs):
    idx = torch.randperm(coords.shape[0])
    coords_shuffled = coords[idx]
    colors_shuffled = colors[idx]

    for i in tqdm(range(0, len(coords), batch_size), desc=f"Epoch {epoch+1}/{epochs}"):
        x_batch = coords_shuffled[i:i+batch_size]
        y_batch = colors_shuffled[i:i+batch_size]

        optimizer.zero_grad()
        y_pred = model(x_batch)
        loss = criterion(y_pred, y_batch)
        loss.backward()
        optimizer.step()

        loss_history.append(loss.item())

```

```

Epoch 41/100: 100%|██████████| 224/224 [00:08<00:00, 25.74it/s]
Epoch 42/100: 100%|██████████| 224/224 [00:07<00:00, 30.64it/s]
Epoch 43/100: 100%|██████████| 224/224 [00:08<00:00, 26.37it/s]
Epoch 44/100: 100%|██████████| 224/224 [00:08<00:00, 26.85it/s]
Epoch 45/100: 100%|██████████| 224/224 [00:07<00:00, 28.71it/s]
Epoch 46/100: 100%|██████████| 224/224 [00:09<00:00, 24.19it/s]
Epoch 47/100: 100%|██████████| 224/224 [00:08<00:00, 25.68it/s]
Epoch 48/100: 100%|██████████| 224/224 [00:07<00:00, 30.14it/s]
Epoch 49/100: 100%|██████████| 224/224 [00:08<00:00, 26.08it/s]
Epoch 50/100: 100%|██████████| 224/224 [00:08<00:00, 27.13it/s]
Epoch 51/100: 100%|██████████| 224/224 [00:07<00:00, 28.94it/s]
Epoch 52/100: 100%|██████████| 224/224 [00:08<00:00, 26.56it/s]

```

Epoch 53/100:	100%		224/224	[00:08<00:00,	25.09it/s]
Epoch 54/100:	100%		224/224	[00:07<00:00,	29.31it/s]
Epoch 55/100:	100%		224/224	[00:08<00:00,	25.10it/s]
Epoch 56/100:	100%		224/224	[00:08<00:00,	27.72it/s]
Epoch 57/100:	100%		224/224	[00:08<00:00,	26.82it/s]
Epoch 58/100:	100%		224/224	[00:08<00:00,	26.10it/s]
Epoch 59/100:	100%		224/224	[00:08<00:00,	25.96it/s]
Epoch 60/100:	100%		224/224	[00:07<00:00,	28.24it/s]
Epoch 61/100:	100%		224/224	[00:08<00:00,	25.34it/s]
Epoch 62/100:	100%		224/224	[00:08<00:00,	26.33it/s]
Epoch 63/100:	100%		224/224	[00:09<00:00,	24.17it/s]
Epoch 64/100:	100%		224/224	[00:08<00:00,	25.13it/s]
Epoch 65/100:	100%		224/224	[00:08<00:00,	25.43it/s]
Epoch 66/100:	100%		224/224	[00:08<00:00,	27.92it/s]
Epoch 67/100:	100%		224/224	[00:08<00:00,	25.31it/s]
Epoch 68/100:	100%		224/224	[00:08<00:00,	25.28it/s]
Epoch 69/100:	100%		224/224	[00:07<00:00,	28.84it/s]
Epoch 70/100:	100%		224/224	[00:08<00:00,	26.29it/s]
Epoch 71/100:	100%		224/224	[00:08<00:00,	25.84it/s]
Epoch 72/100:	100%		224/224	[00:07<00:00,	28.43it/s]
Epoch 73/100:	100%		224/224	[00:08<00:00,	24.94it/s]
Epoch 74/100:	100%		224/224	[00:07<00:00,	28.03it/s]
Epoch 75/100:	100%		224/224	[00:08<00:00,	26.64it/s]
Epoch 76/100:	100%		224/224	[00:08<00:00,	26.21it/s]
Epoch 77/100:	100%		224/224	[00:08<00:00,	26.28it/s]
Epoch 78/100:	100%		224/224	[00:07<00:00,	28.22it/s]
Epoch 79/100:	100%		224/224	[00:10<00:00,	22.34it/s]
Epoch 80/100:	100%		224/224	[00:07<00:00,	28.56it/s]
Epoch 81/100:	100%		224/224	[00:08<00:00,	25.38it/s]
Epoch 82/100:	100%		224/224	[00:08<00:00,	25.52it/s]
Epoch 83/100:	100%		224/224	[00:07<00:00,	28.30it/s]
Epoch 84/100:	100%		224/224	[00:08<00:00,	25.24it/s]
Epoch 85/100:	100%		224/224	[00:09<00:00,	24.82it/s]
Epoch 86/100:	100%		224/224	[00:08<00:00,	26.32it/s]
Epoch 87/100:	100%		224/224	[00:08<00:00,	26.79it/s]
Epoch 88/100:	100%		224/224	[00:08<00:00,	25.17it/s]
Epoch 89/100:	100%		224/224	[00:08<00:00,	27.22it/s]
Epoch 90/100:	100%		224/224	[00:08<00:00,	26.29it/s]
Epoch 91/100:	100%		224/224	[00:08<00:00,	26.05it/s]

```
Epoch 92/100: 100%|██████████| 224/224 [00:08<00:00, 26.46it/s]
Epoch 93/100: 100%|██████████| 224/224 [00:08<00:00, 25.76it/s]
Epoch 94/100: 100%|██████████| 224/224 [00:08<00:00, 25.73it/s]
Epoch 95/100: 100%|██████████| 224/224 [00:08<00:00, 25.25it/s]
Epoch 96/100: 100%|██████████| 224/224 [00:08<00:00, 24.97it/s]
Epoch 97/100: 100%|██████████| 224/224 [00:08<00:00, 25.11it/s]
Epoch 98/100: 100%|██████████| 224/224 [00:07<00:00, 29.31it/s]
Epoch 99/100: 100%|██████████| 224/224 [00:08<00:00, 25.37it/s]
Epoch 100/100: 100%|██████████| 224/224 [00:08<00:00, 26.78it/s]
```

5.3 [1 point] Once the network is trained, predict RGB color at each pixel location, visualize generated image. Also visualize difference between generated image and original image.

```
with torch.no_grad():
    pred = model(coords).reshape(H, W, 3)
    predict_img = pred.numpy()

import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))

ax1.imshow(img)
ax1.set_title('original')
ax1.axis('off')

ax2.imshow(predict_img)
ax2.set_title('predict')
ax2.axis('off')

plt.tight_layout()

plt.show()
```

original



predict



